

AGRIAI 2.0

INTELLIGENT FARMING ASSISTANT FOR GHANA

Volume V. Security Threat Model

What is already locked, what can still bite, and how Thaddeus Tagoe designed the trade offs

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Computer Science, University of Ghana

Founder, designer, and sole engineer of AgriAI

Repository: ThaddeusHackz/agriai-v2

Document date: 18 August 2026

Version: 2.0.0 | Classification: Internal forensic architecture pack

This dossier is a forensic engineering record. It shows how AgriAI was built, which source files make the AI work, why answers still arrive when a vendor is down, and how the public and admin APIs stay reliable. Every claim is verified against the live codebase on 18 August 2026. No em dash character is used. The sole creator of every line of product code and of this dossier is Thaddeus Nii Teiko Tagoe.

Forensic scan executed on branch arena/01a0159e-agriai-v2, commit 9914572, Next.js 16.2.10, React 19.2.4, TypeScript 5, Tailwind CSS 4. Evidence tables in this pack list real file paths, real line counts, and real provider contracts. Reading alone will not make the system clear, the technical wiring will.

1. Scope and method

This volume is an honest forensic review of AgriAI 2.0 as it exists on disk in ThaddeusHackz/agriai-v2. It is not a live penetration test against a host. It reads lib/auth.ts, next.config.ts, app/api/**/route.ts validators, lib/env.ts secret handling, lib/db.ts storage, and the trust diagram, then names real risks a Ghana deployment will meet and rates their residual risk. All recommendations are ordered so the sole operator can act.

2. Trust boundaries diagrammed in words

- Zone 1 Browser: farmer or attacker on phone. May only call AgriAI HTTP routes under app/api/*. No vendor key is ever sent in NEXT_PUBLIC_ except NEXT_PUBLIC_SITE_URL. Anything the browser sends is untrusted.
- Zone 2 Next.js Node runtime: holds GEMINI, Cloudflare, Tavily, OpenWeather, ElevenLabs, Unsplash, DATABASE_URL, ADMIN_EMAIL and ADMIN_PASSWORD. This is the only place secrets exist, read via lib/env.ts firstEnv.
- Zone 3 Vendors: Google Gemini, Cloudflare Workers AI, Tavily, OpenWeather, ElevenLabs, Unsplash, Wikipedia, DuckDuckGo. Output from these is treated as untrusted text that can contain injected instructions. AgriAI never executes it, it only grounds or renders it as markdown after react-markdown sanitization.

- Zone 4 Storage: data/db.json (gitignored, atomic rename) and PostgreSQL agriai_state id 1 JSONB. Holds chats that may contain farm locations and phone numbers, subscribers emails, contacts, sessions with IP and UA, and bcrypt hashes. Disk and DB are the crown jewels.
- Zone 5 Admin identity: cookie agriai_session holds a 32 byte hex token, httpOnly sameSite lax secure in production path /. Role is admin or editor per AdminUser. No JWT, no OAuth on this branch.

3. Controls that already work (forensic evidence that Thaddeus built security in)

- Passwords hashed with bcryptjs cost 10. Hash call is hashSync(password, 10) in lib/auth.ts and lib/db.ts. publicUser strips passwordHash before any JSON leaves the server.
- Session token is crypto.randomBytes 32 hex (64 chars). Created in startSession, pushed into sessions with createdAt plus expiresAt now plus 7 days, ip truncated, ua sliced to 200. Expired sessions pruned on login.
- Cookie agriai_session is set via withSessionCookie helper with httpOnly true, sameSite lax, secure when NODE_ENV production, path /, maxAge 7 days when remember true else 8 hours. clearSessionCookie sets maxAge 0.
- Login rate limit: Map attempts keyed by clientIp (x-forwarded-for first hop or x-real-ip else unknown), MAX_ATTEMPTS 5, WINDOW_MS 15 minutes, returns 429 Too many failed attempts. Failed login does not reveal whether email exists, same message for both cases.
- Route validation: chat message sliced to 2000, history filtered to last 10 each 0 to 4000, language and mode regex whitelisted, image 4.5 MB, audio 25 MB, TTS text 1500, generate prompt 500 plus SAFE_SUBJECTS gate.
- Secret hygiene: lib/env.ts firstEnv trims whitespace, strips leading and trailing quotes, rejects blank and YOUR_ and CHANGE_ME and placeholder, never logs secret values, and never returns them in /api/health or /api/config.
- Headers: next.config.ts adds X-Content-Type-Options nosniff and Referrer-Policy strict-origin-when-cross-origin for every route. Health route /api/health reports provider configured and live flags but not keys.
- Storage hygiene: data/db.json.tmp then rename atomic, schedulePostgresSave debounce, data folder gitignored, default admin only when file missing.
- Reset and user CRUD plus knowledge and prices writes all require getSessionUser to be non null.

Evidence: lib/auth.ts lines 30 to 120 contain bcrypt, attempts Map, createSessionToken, startSession, getSessionUser, withSessionCookie. next.config.ts lines 4 to 20 contain allowedDevOrigins plus headers. grep -n "passwordHash" lib/db.ts shows bcrypt on seed.

4. Threat catalog (ten threats that matter for a farming product)

4.1 Prompt injection and jailbreak via message or via search results

Attack: A farmer or a script puts hidden instructions in the message or in a page that Tavily then injects into the system block as CURRENT WEB SEARCH RESULTS. The model may be told to ignore Ghana agronomy rules, to invent pesticide doses, to output markdown that leaks the system prompt, or to echo a hidden link as a citation. The route concatenates search text into the system string with no stripping of lines that start with Instruction. Impact is advice quality or brand. Mitigation present: citation markers, short maxTokens, temperature from admin, mode prompts that tell the model to cite only when search context present. No allow list strips instruction lines. Residual risk: high for advice correctness on expert doses, medium for brand if jailbreak leaks prompt. Operator action: keep expert mode for trained staff, show a disclaimer that chemical rates must be confirmed with MoFA, and consider a filter that drops search lines containing ignore previous instructions before they enter the prompt.

4.2 Cost abuse and denial of wallet (unauthenticated AI spend)

Attack: POST /api/chat, /api/vision, /api/generate, /api/transcribe, /api/tts have no farmer login and no per IP quota on the Node process. An open Render URL can be scripted to send 2000 char chats, 4 MB images, 500 char prompts, 25 MB audio until Gemini, Flux, or ElevenLabs bills spike. Chat also writes every turn to disk and then Postgres so storage grows. Mitigation present: 60 second maxDuration, byte caps, SAFE_SUBJECTS gate on generate. Residual risk: high on a public hostname with an LLM key that is pay as you go. Operator action: put a reverse proxy rate limit (Cloudflare WAF or nginx limit_req) in front of /api/chat, /api/vision, /api/generate, /api/tts, /api/transcribe, add per IP burst limits, and a site wide daily AI budget that flips chat to local demo when exceeded.

4.3 Admin session theft via XSS or disk leak

Attack: sameSite lax blocks most CSRF cookie sends from evil sites on POST in modern browsers, but any XSS on the AgriAI origin (for example through a malicious knowledge answer rendered as markdown that an admin views) would still be able to fetch /api/admin/* as the victim even though httpOnly blocks reading the cookie. Sessions live as plain tokens in data/db.json and Postgres JSON, so a stolen file is a stolen login list. There is no CSRF token pair. Mitigation present: httpOnly cookie, SameSite lax, react-markdown rendering (not dangerously setting innerHTML), bcrypt passwords. Residual risk: medium, because markdown XSS in remark-gfm plus raw HTML could be enough. Operator action: enforce HTTPS only, keep markdown allowed HTML off, add CSP header, move sessions to signed cookies or to a separate session store, and rotate staff passwords after any disk backup leak.

4.4 Default and seed credentials on first boot

Attack: defaultDB in lib/db.ts creates an admin user with email from ADMIN_EMAIL or admin@agriai.gh and password from ADMIN_PASSWORD or AgriAI@2026Admin when data/db.json does not exist. That password is well known from the source. If the operator forgets to set env before first boot on Render, an attacker can log in with the known pair and own the panel, change prompts, scrape messages. README already says change the password in Admin Users immediately. Some deploys may not. Residual risk: critical on any public demo that skipped env. Operator action: set a long random ADMIN_PASSWORD in Render env before the first process start, then log in and change it again in the panel, and disable the fallback string in code before a ministry rollout (throw if ADMIN_PASSWORD not set when NODE_ENV production).

4.5 Rate limit bypass on multi instance or via header spoof

Attack: attempts is an in-memory Map keyed by clientIp which reads x-forwarded-for split comma first hop, or x-real-ip else unknown. On multi instance Render the limit lives per process so five per instance becomes ten or fifteen globally. If a reverse proxy is misconfigured to trust all X-Forwarded-For hops, an attacker can spoof a fresh IP per request. Token brute force then becomes easy. Mitigation present: 5 per 15 minutes per instance. Residual risk: medium for brute force, high for targeted attack on edge. Operator action: rely on the platform edge rate limit (Render or Cloudflare) not the in-memory Map for production, or move limits to Postgres or Redis, and trust only the first hop inserted by the platform.

4.6 Sensitive data in the document store and in backups

Attack: chats may contain farm locations, phone numbers, family names, or distress messages about crop loss. Subscribers and contacts are emails, contacts contain free text that may be sensitive. Sessions include IP and user agent. The JSON file is gitignored which is correct, but a backup tarball, a world readable Render disk snapshot, or an admin CSV export sent over email would expose them. The Postgres mirror agriai_state holds the same PII in one row. Impact is privacy under Ghana Data Protection Act. Residual risk: high if backups are casual or the admin Messages tab is open to many staff. Operator action: encrypt the host disk at rest, restrict who can open /admin/messages, publish a 90 day chat retention job (delete chats older than 90 days via cron), and avoid emailing CSV exports without password.

4.7 SSRF and outbound fetch abuse

Attack: search, weather, Cloudflare, Tavily, Wikipedia, DuckDuckGo, ElevenLabs, Unsplash all use hard coded vendor URLs with query params, not farmer supplied URLs. generate does not fetch a farmer URL, it only posts a prompt. There is no fetch arbitrary URL feature in this codebase, so classic SSRF to 169.254.169.254 metadata is not reachable. If a future feature scrapes any link a farmer pastes, that claim would change. Residual risk: low on this branch, medium if link scraping is added later. Operator action: keep farmer URLs as display only, and if scraping is ever added put a URL allow list plus no private IP range (10/8, 172.16/12, 192.168/16, metadata IP).

4.8 File and vision abuse (quota burn and CPU)

Attack: Vision accepts a huge base64 string inside JSON, not as multipart, so nginx limit may not catch it before Next.js buffers it. Caps exist at 4.5 MB but a flood of 4 MB posts still burns CPU for base64 decode plus Gemini quota. There is no virus scan because bytes are sent straight to Gemini, not saved as shared gallery files, so malware on disk is low risk. CPU and quota are the risk. Residual risk: medium for cost, low for malware. Operator action: keep 4.5 MB cap, add per IP quota on /api/vision, and in front of transcription cap 25 MB add multipart validation rejected count.

4.9 Health probe information leak (recognizance)

Attack: GET /api/health is public with no auth and reports { ok, version 2.0.0, providers { gemini configured live, cloudflare configured, openweather, tavily, elevenlabs, unsplash, database }, aliases }. When gemini is configured it even does a live ping with generateContent model gemini-2.0-flash prompt Reply with the single word OK max tokens 8 and reports live true or error first 220 chars. This helps Thaddeus debug Render but also tells an attacker which wallet to drain or which provider is down. Residual risk: low to medium for targeted wallet attacks. Operator action: lock /api/health to the platform checker IP or strip the live error string to a boolean in production, keep it open only as { ok true } plus version.

4.10 Missing browser hardening headers (clickjacking and CSP)

Observation: next.config.ts sets nosniff and referrer policy but not Content-Security-Policy, not Permissions-Policy, not X-Frame-Options, not CSP frame-ancestors. An attacker can try to frame /admin/login in an iframe on an HTTP page to trick staff (clickjacking) on older browsers that do not block framing by default. XSS scope also widens without CSP. Residual risk: low on modern Chrome with fetch metadata but higher on embedded web views farmers may use. Operator action: add header frame-ancestors 'self' or X-Frame-Options DENY, add CSP default-src 'self' script-src 'self' style-src 'self' plus Google Fonts domains only, once the inline style inventory is complete.

5. AI quality abuse that is not a traditional vulnerability but a safety issue

- Medical or pesticide overconfidence: expert mode can state kg per ha figures that the underlying LLM invented, not that agronomy peer review confirmed. The UI keeps demo and live disclaimers, but a farmer may still follow a rate. Advice must be presented as guidance to confirm with the district MoFA officer.
- Language mismatch: a Twi request can still receive English if the model ignores languageInstruction. Farmer safety depends on the farmer noticing. A language badge helps but does not fix the model.
- Citation theater: if search succeeds but returns a wrong snippet, the model can still cite it as [1]. Citations are not proof of truth, they are proof of fetch.
- Image studio SAFE_SUBJECTS is substring based so creative spelling or concatenation can slip past (e.g. maizeevil). Keep Flux account limits tight so the cost cannot spike even if the gate is passed.

6. Recommended hardening order for the sole operator (do these in sequence)

- 1 Set a unique ADMIN_PASSWORD in Render env before first boot, then change again in the panel via Admin Users.
- 2 Hold all secrets only in Render secret store and in .env.local locally, never commit .env.local, never paste keys into chat logs.
- 3 Put edge rate limits (Cloudflare WAF or nginx limit_req) on POST /api/chat, /api/vision, /api/generate, /api/tts, /api/transcribe with burst and daily caps.
- 4 Restrict /admin and /api/admin to known staff IPs or to Google OAuth if the panel must be public, else keep it off the public site and use a VPN.
- 5 Add Content-Security-Policy and frame-ancestors 'self' to next.config.ts after auditing inline styles and fonts.
- 6 Move sessions from db.json to a signed cookie or to Redis so a stolen file does not clone logins and limits can be shared across instances.
- 7 Redact /api/health error strings to boolean in production and require auth for detailed provider checks.
- 8 Publish and enforce a 90 day chat retention cron that deletes old conversations from both db.json and Postgres, and document it for Data Protection compliance.

7. Forensic verdict on security (what Thaddeus built versus what remains)

AgriAI was engineered as a farming product that must stay up for farmers, not as a bank vault. Auth with bcrypt cost 10, httpOnly sameSite lax secure cookies, rate limited login 5 per 15, input caps on every AI byte, SAFE_SUBJECTS gate on images, placeholder rejection on env, nosniff header, and key isolation via lib/env.ts are real engineering by Thaddeus Tagoe that pass a basic forensic audit. The largest remaining holes are unauthenticated AI spend (wallet attack), seed password discipline (default string), in-memory limits that do not survive multi instance, and PII in one document that needs a retention rule and disk encryption. Close those four before a ministry scale rollout and the rest of the findings become routine hardening.